VELOOP REWARDS - Logical Data Model

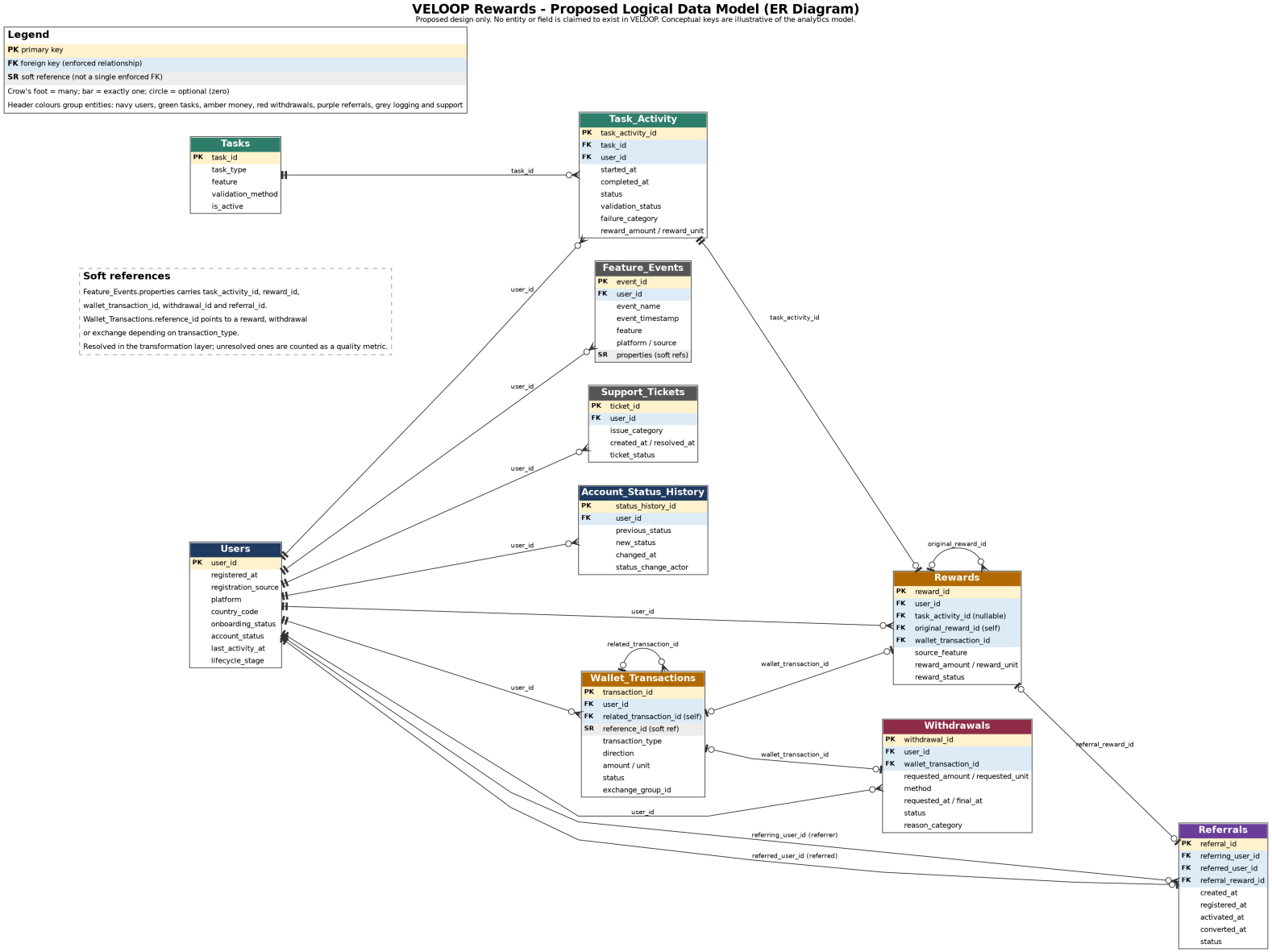
Data Analyst Internship, Task 01 | Work Package H

Authenticity notice: This model is a **Proposed** design. No entity, key or relationship is claimed to exist in VELOOP, and no simulated or real VELOOP statistics appear here.

1. Summary

- **10 entities**, 16 foreign keys and 20 documented relationships (Sections 3 to 5). Users is the anchor: every other entity joins back to it through user_id.
- **One many-to-many relationship**: Users and Tasks, resolved by Task_Activity, which holds each attempt's own data.
- **Two self-references** carry reversals: Rewards.original_reward_id and Wallet_Transactions.related_transaction_id. Originals are never edited.
- **Two soft references** are deliberate: Feature_Events.properties and Wallet_Transactions.reference_id. They cannot be a single enforced foreign key (see Section 5).
- **10 integrity rules** turn the model into checks that feed the data quality framework and the relationship tests on the simulated dataset.
- Task_Activity.reward_id (Data Dictionary) is a convenience copy. The authoritative link is Rewards.task_activity_id. IR04 checks that they match; engineering may drop the copy.

2. ER diagram (H4)



A vector version for zooming is in 10_ER_Diagram.pdf.

3. Primary keys (H1)

Entity	Primary Key	Type	Grain (one row per)	Key Rules	Status
Users	user_id	string	user account	Opaque; unique; immutable; never derived from personal data; never reused	To Be Verified
Account_Status_History	status_history_id	string	account status change	Unique; one row per change; append-only	Proposed
Tasks	task_id	string	task in the catalogue	Unique; stays the same when a task's title or reward changes	To Be Verified
Task_Activity	task_activity_id	string	task attempt by one user	Unique per attempt; repeat attempts get new ids	Proposed
Rewards	reward_id	string	reward	Unique; a reversal keeps the original reward_id and links to it	Proposed
Wallet_Transactions	transaction_id	string	ledger entry	Unique; server-generated; writes are idempotent; entries are never edited	To Be Verified
Withdrawals	withdrawal_id	string	withdrawal request	Unique; a rejected request is never reopened, a new request gets a new id	To Be Verified
Referrals	referral_id	string	referral relationship	Unique; one row per referrer-referred pair	Proposed
Feature_Events	event_id	string	tracked event occurrence	Unique; a retry reuses the same event_id	Proposed
Support_Tickets	ticket_id	string	support request	Unique	To Be Verified

4. Foreign keys (H2)

Child Entity	Foreign Key	Parent Entity	Parent Key	Nullable?	Handling Rule	Status
Account_Status_History	user_id	Users	user_id	No	Parent is never hard-deleted	To Be Verified
Task_Activity	user_id	Users	user_id	No	Parent is never hard-deleted	To Be Verified
Task_Activity	task_id	Tasks	task_id	No	Retired tasks are flagged is_active = false, not deleted	To Be Verified
Rewards	user_id	Users	user_id	No	Parent is never hard-deleted	To Be Verified
Rewards	task_activity_id	Task_Activity	task_activity_id	Yes	Null for rewards not earned through a task (spin, daily reward, referral)	Proposed
Rewards	original_reward_id	Rewards	reward_id	Yes	Self-reference for reversal and correction; not equal to reward_id; no chains	Proposed
Rewards	wallet_transaction_id	Wallet_Transactions	transaction_id	Yes	Null while pending; required once credited	Proposed
Wallet_Transactions	user_id	Users	user_id	No	Parent is never hard-deleted	To Be Verified
Wallet_Transactions	related_transaction_id	Wallet_Transactions	transaction_id	Yes	Required for reversal and adjustment; not equal to transaction_id; target cannot itself be a reversal	Proposed
Withdrawals	user_id	Users	user_id	No	Parent is never hard-deleted	To Be Verified
Withdrawals	wallet_transaction_id	Wallet_Transactions	transaction_id	Yes	The withdrawal_debit entry; exactly one active debit per withdrawal	Proposed
Referrals	referring_user_id	Users	user_id	No	Role: referrer	To Be Verified
Referrals	referred_user_id	Users	user_id	Yes	Role: referred; empty until the invited user registers; must differ from referring_user_id	To Be Verified
Referrals	referral_reward_id	Rewards	reward_id	Yes	Null until a reward is given	To Be Verified

Child Entity	Foreign Key	Parent Entity	Parent Key	Nullable?	Handling Rule	Status
Feature_Events	user_id	Users	user_id	No	Raw layer: kept even if the user is later closed	To Be Verified
Support_Tickets	user_id	Users	user_id	No	Parent is never hard-deleted	To Be Verified

5. Relationship documentation (H3)

Cardinality reads parent : child. '0..N' means zero or more, '0..1' means zero or one.

f	Type	Parent	Child	Cardinality	Via	Explanation	Status
1	one-to-many	Users	Account_Status_History	1 : 0..N	user_id	One user can have many status changes	Proposed
2	one-to-many	Users	Task_Activity	1 : 0..N	user_id	One user can attempt many tasks, many times	Proposed
3	one-to-many	Tasks	Task_Activity	1 : 0..N	task_id	One task can be attempted many times by many users	Proposed
4	many-to-many (resolved)	Users	Tasks	N : M	Task_Activity	Many users attempt many tasks. Task_Activity is the associative entity and carries the attempt's own data (time, status, reward)	Proposed
5	one-to-many	Users	Rewards	1 : 0..N	user_id	One user receives many rewards	Proposed
6	one-to-one (optional)	Task_Activity	Rewards	1 : 0..1	task_activity_id	An attempt produces at most one reward. Rewards.task_activity_id is the authoritative link	Proposed
7	self-reference	Rewards	Rewards	1 : 0..N	original_reward_id	Reversals and corrections point to the reward they change	Proposed
8	transaction	Wallet_Transactions	Rewards	0..1 : 0..1	wallet_transaction_id	A credited reward has one wallet credit entry; adjustments have no reward	Proposed
9	one-to-many	Users	Wallet_Transactions	1 : 0..N	user_id	One user owns many ledger entries	Proposed
0	self-reference (transaction)	Wallet_Transactions	Wallet_Transactions	1 : 0..N	related_transaction_id	Reversals and adjustments point to the original entry; the original is never edited	Proposed
1	transaction	Wallet_Transactions	Withdrawals	0..1 : 0..1	wallet_transaction_id	A withdrawal has one debit entry; a later reversal is a separate linked entry	Proposed
2	one-to-many	Users	Withdrawals	1 : 0..N	user_id	One user can make many withdrawal requests	Proposed
3	one-to-many	Users	Referrals	1 : 0..N	referring_user_id	One user can invite many people (referrer role)	To Be Verified
4	one-to-one (optional)	Users	Referrals	1 : 0..1	referred_user_id	A user can have at most one referrer (referred role); to be verified with the product team	To Be Verified
5	one-to-one (optional)	Rewards	Referrals	0..1 : 0..1	referral_reward_id	A referral can earn one reward	To Be Verified
6	event	Users	Feature_Events	1 : 0..N	user_id	Every event belongs to one user	Proposed
7	event (soft)	Task_Activity, Rewards, Wallet_Transactions, Withdrawals, Referrals	Feature_Events	1 : 0..N	properties	One business record produces several events over its life, for example a withdrawal produces requested, processing and completed. Linked through keys inside properties, not an enforced FK	Proposed
8	transaction (soft)	Rewards, Withdrawals, exchanges	Wallet_Transactions	1 : 0..N	reference_id	reference_id points to a different table depending on transaction_type, so it cannot be one FK; check it per type	Proposed
9	transaction (grouping)	(none)	Wallet_Transactions	2 : 1 group	exchange_group_id	The two entries of one exchange share a group key. No separate Exchange entity is justified yet	Proposed
0	one-to-many	Users	Support_Tickets	1 : 0..N	user_id	One user can raise many support requests	To Be Verified

6. Integrity rules

Rule	Check	Why	Priority
IR01	Every foreign key value exists in its parent table (no orphans)	Orphans break joins and hide activity	Critical
IR02	Each credited reward has exactly one reward_credit wallet entry with reference_id = reward_id	Rewards and wallet must agree	Critical
IR03	Each completed withdrawal has exactly one active withdrawal_debit entry	Prevents paying without deducting, or deducting twice	Critical
IR04	Rewards.task_activity_id agrees with Task_Activity.reward_id wherever both exist	Two copies of one link must match	High
IR05	original_reward_id and related_transaction_id never point to themselves and never form chains	Reversal of a reversal is a data error	High
IR06	referred_user_id differs from referring_user_id; a user has at most one referrer	Self-referral and double referral are integrity errors	High
IR07	Exchange groups have exactly one debit and one credit	Prevents one-sided exchanges	High
IR08	Users are never hard-deleted; closed accounts keep their user_id	History must keep joining	High
IR09	Child timestamps are not earlier than the parent's creation (for example Task_Activity.started_at >= Users.registered_at)	Impossible order signals bad data	Medium
IR10	Keys found inside event properties resolve to a business record in the transformation layer; the unresolved share is tracked	Soft references need a measured quality level	Medium

7. Relationship tests planned for the simulated dataset (Work Package J2)

Parent key	Child key to test
Users.user_id	Task_Activity.user_id
Users.user_id	Rewards.user_id
Users.user_id	Wallet_Transactions.user_id
Users.user_id	Withdrawals.user_id
Users.user_id	Referrals.referring_user_id and referred_user_id
Users.user_id	Feature_Events.user_id
Tasks.task_id	Task_Activity.task_id
Task_Activity.task_activity_id	Rewards.task_activity_id
Rewards.reward_id	Wallet_Transactions.reference_id (reward_credit)
Withdrawals.withdrawal_id	Wallet_Transactions.reference_id (withdrawal_debit)

8. Notes and open questions

- This is a logical model: it says what the entities and keys are, not which database stores them.
- Feature_Events is a raw logging table. Its links to business records are soft by design, so a delayed or missing event never blocks a business write.
- Can a user have only one referrer? (A010)
- Is the wallet an append-only ledger today? (A005)
- Does a pending reward already have its own row, or only a credited reward? (A020)

- Is there a separate redemption or exchange record in the product that would need its own entity?
- Are task attempts stored today, or only completions? (A001)

Next: Work Package I - KPI Catalogue (30+ KPIs).